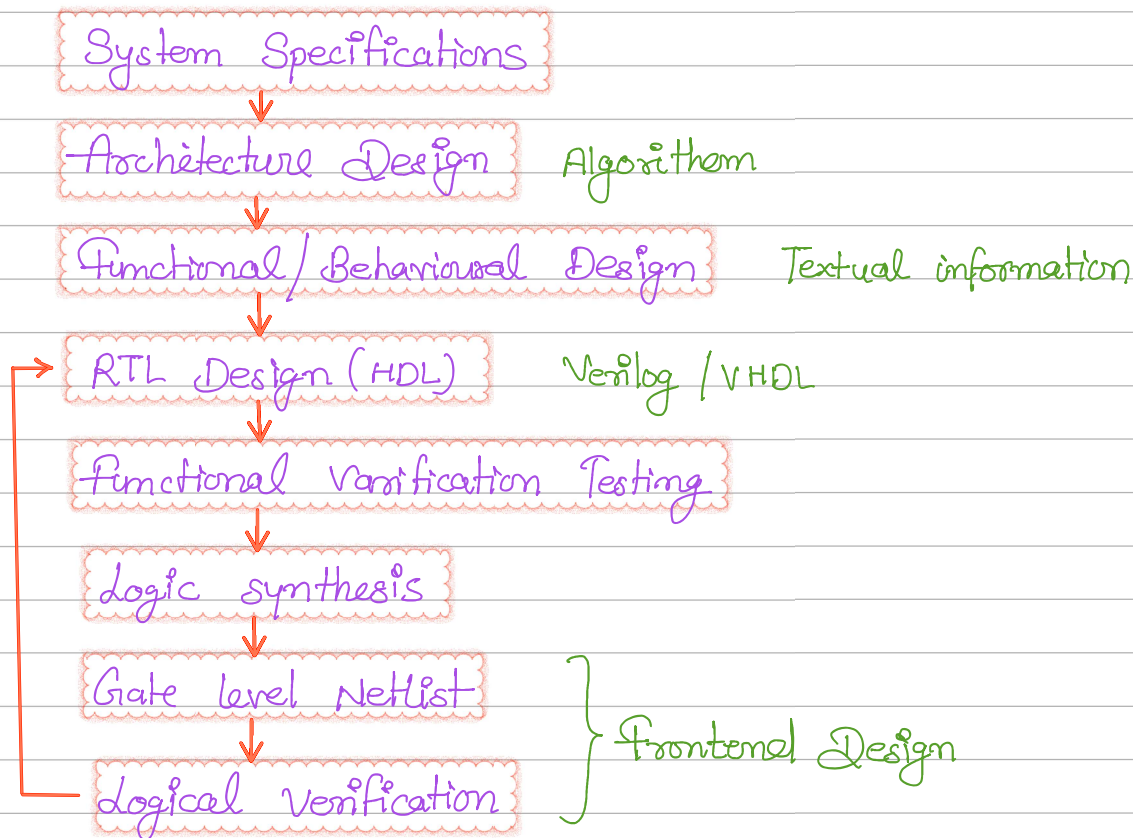


Programming

→ Verilog
→ VHDL

Prerequisite : Digital electronics

IC Design flow



★ In logic synthesis tool the is convert which is a gate level netlist- the tools used is design compiler from synopsys in Gurus from cadence.

The inps required ① the RTL code , library files and sdc (synopsys design constrain files) are required.

It happens in three stages —

1. Translate
2. Mapping
3. Optimise

(i) Translate :- In translate, the tool converts the high level ^{RTL code} to its corresponding logic gate structure.

(ii) mapping :- Where the tool maps the logic gates with required technology in which we want to design, ex: 7nm, 14nm etc.

(iii) Optimise :- the tool does optimization in terms of power, area, voltage and timing in order to get best QOR (Quality of result)

Design ~~error~~ test with also done at this stage by DFT team.

Logic Verification / Testing :-

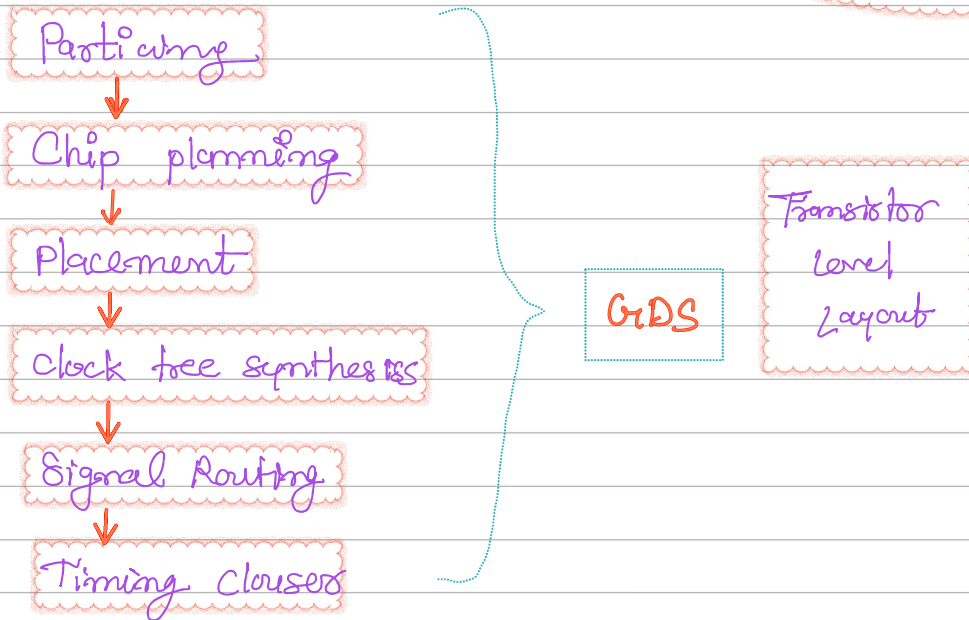
Identification wave run to check the error/bugs.

Logical verification/Testing

EDA Tools

Physical Design

ICC2 from Synopsys
Invenvus from Cadence



Here the physical design at the gate level is converted to GDs (Graphic Data Stream).

The physical design has a stage which is called PNR (Place & Route) where the placement & routing of all the instances present in the netlist are placed in a defined core area in such a way that it should meet the design's rule and timing requirement.

Physical Layout



Physical Layout verification & signoff

-fabrication

Packaging & Testing

chip

The obtained GDS file is sent for verification & sign-off where diff verification are performed ex- DRC, LVS, ERC. there after timing, signals cross & power analysis.

The GDS file after the verification is sent to the foundry for fabrication which is also called as tape out. the mask is generated in foundry & chip is fabricated.

the fabricated chip is package to protect against external damage & then again tested for any bugs before releasing external market.

Dated: 27/08/24 - Tuesday

J.E. Lilienfeld → 1960 : Innovator of IC

→ what pattern we follow while reducing the size of transistors?
→ Y-chart??

28/08/2024 wednesday

Design Methodology

FSM - Finite State M/c

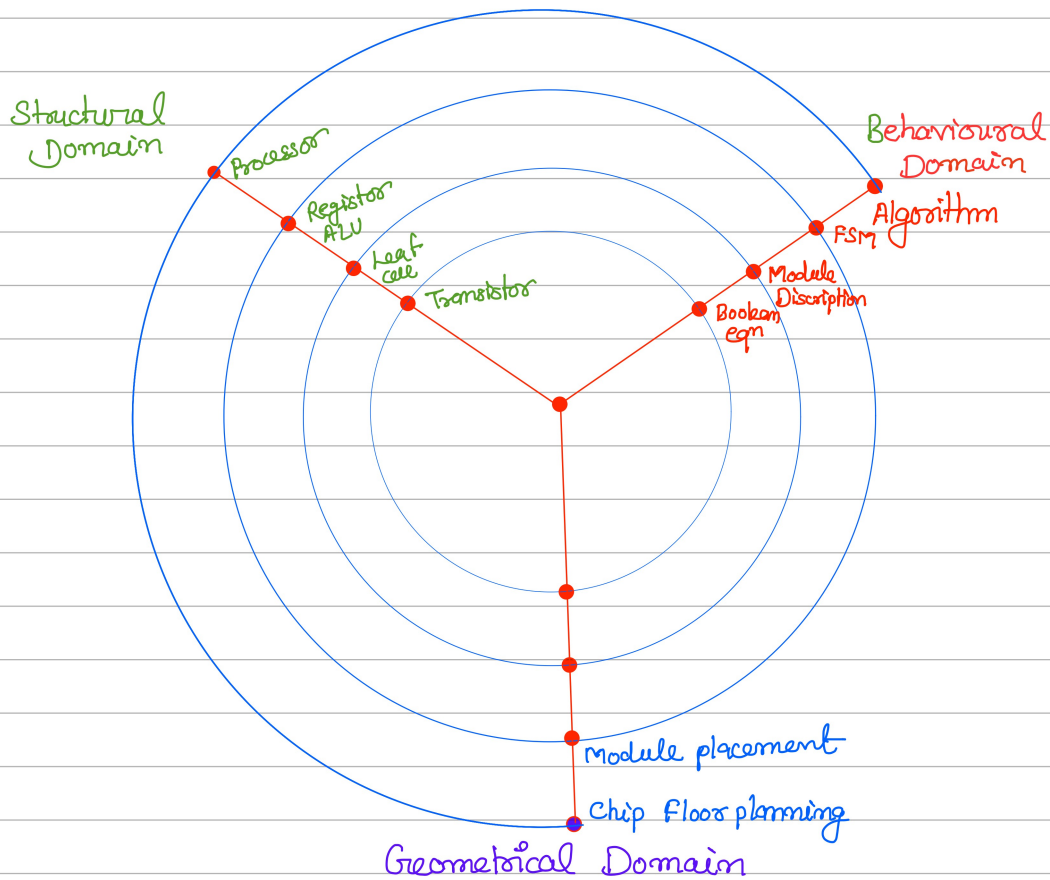
4-Bit Adder

- ↳ Top-Down Design methodology
- ↳ Bottom-Up design methodology

Y-chart

↳ Method to represent design methodology

y-chart



5 quizzes & 2 projects & 1 Assignment

1 quiz → 2nd week sept

Date :- 02/09/2024

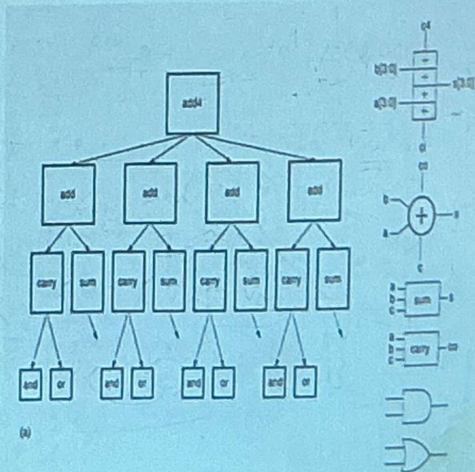
* Design Hierarchy :-

Design Hierarchy

- The use of hierarchy, or the "divide and conquer" technique, involves dividing a module into sub-modules and then repeating this operation on the sub-modules until the complexity of the smaller parts becomes manageable.
- This approach is very similar to the software case where large programs are split into smaller and smaller sections until simple subroutines, with well-defined functions and interfaces, can be written.
- We know that the design of the VLSI chip can be represented in three domains.
- Correspondingly, a hierarchy structure can be described in each domain separately.
- However, it is important for the simplicity of the design that the hierarchies in different domains can be easily mapped into each other.

Example: A CMOS four-bit adder

- The adder can be progressively decomposed into one-bit adders, separate carry and sum circuits, and finally, individual logic gates.
- At this lower level of the hierarchy, the design of a simple circuit realizing a well-designed Boolean function is much easier to handle than at the higher levels.



Design Hierarchy

- In the Physical domain, partitioning a complex system with its various functional blocks will provide valuable guidance for realizing these blocks on a chip.
- Obviously, each submodule's approximate shape and size (area) should be estimated to provide a helpful floor plan.

Concepts of Regularity

- The hierarchical design approach reduces the design complexity by dividing the extensive system into sub-modules.
- Usually, other design concepts and design approaches are also needed to simplify the process.
- Regularity means that the hierarchical decomposition of the large system should result in not only simple but also similar blocks, as much as possible.
- A good example of regularity is the design of array structures consisting of identical cells – such as a parallel multiplication array, PAL, PLA, and PROM.
- Regularity can exist at all levels of abstraction:
 - ✓ At the transistor level, uniformly sized transistors simplify the design.
 - ✓ At the logic level, identical gate structures can be used, among others.
- If the designer has a small library of well-defined and well-characterized basic building blocks, several functions can be constructed using this principle.
- Regularity usually reduces the number of different modules that need to be designed and verified, at all levels of abstraction.

Concepts of modularity

- Modularity in design means that the various functional blocks which make up the larger system must have well-defined functions and interfaces.
- Modularity allowed that each block or module can be designed relatively independently from each other, since there is no ambiguity about the function and the signal interface of these blocks.
- All the blocks can be combined with ease at the end of the design process, to form the large system.
- The concept of modularity enables the parallelization of the design process, reduces development time.
- It also allows the use of generic modules in various designs- the well defined functionality and signal interface allow plug-and-play design.

Concepts of Locality

- By defining well-characterized interfaces for each module in the system, we effectively ensure that the internals of each module become unimportant to the exterior modules.
- Internal details remain at the local level.
- The concept of locality also ensure that connections are mostly between neighboring modules, avoiding long-distance connections as much as possible.
- This last point is extremely important for avoiding excessive interconnect delays.
- Time-critical operations should be performed locally, without the need to access distant modules or signals.
- If necessary, replication of some logic may solve this problem in large system architectures.

Design Methodology

- Full Custom Design
- Semi Custom Design
 - Standard Cell
 - Gate Array
 - PLD's
 - ✓ SPLD
 - ✓ CPLD
 - ✓ FPGA

Full Custom Design

- Full-custom design is a methodology for designing IC's wherein the layout, geometry, orientation and placement of every transistor is done individually by the designer.
- Design productivity is very low – typically few tens of transistors per day per design.
- Full-custom design potentially maximizes the performance of the chip, and minimizes its area, but is extremely labor-intensive to implement.
- Full-custom design is limited to ICs that are to be fabricated in extremely high volumes, notably certain microprocessors (Intel Pentium µp chip) and a small number of ASICs.
- The main factor affecting the design and production of ASICs is the high cost of mask sets and the requisite EDA design tools.
- The mask sets are required in order to transfer the ASIC designs onto the wafer.

Semi-Custom Design : Standard Cell

- All commonly used logic cells are developed, characterized and stored in the standard cell library.
- Library may contain few hundred cells like inverters, NAND, NOR, gates, complex AOI gates, DFF, counters.....
- Each gate type can be implemented in several versions to provide adequate driving capability(fan-out) – inverter gate can have standard size, double size and quadruple size. Chip designer can choose one of it, to meet high current, speed and layout density requirements.
- Designer sends schematic to the fabricator who prepares mask, if the cells are from his library.
- Hence it is captivated by the company. Larger the library, larger will be the cost.
- Standard cell guarantees that design will work.

Semi-Custom Design : ~~Standard Cell~~ ^{GATE Array}

- A gate array circuit is a prefabricated chip/circuit with no particular function, in which transistors and other active devices are placed (unconnected) at regular predefined positions and manufactured on a wafer, usually called as master slice.
- Only masks for metallization needs to be created.
- Interconnection pattern for basic logic gates can be stored in a library for further chip customization.
- This layer is analogous to the copper layer of a printed circuit board (PCB).
- Gate array master slices are usually prefabricated and stockpiled in large quantities regardless of customer orders.

Semi-Custom: Gate array

□ Advantages:

- Less time to market. The design and fabrication according to the individual customer specifications may be finished in a shorter time compared with standard cell or full custom design.
- This approach reduces the mask cost since fewer custom masks need to be produced.

□ Disadvantages:

- Chip size is fixed.
- Lower efficiency as most of the transistors may not be in use.

Semi-Custom Design : PLD's

- PLD is an IC, designed to be configured by the customer or designer after manufacturing—hence "programmable".
- The PLD configuration is specified using a hardware description language (HDL), similar to that used for an application-specific integrated circuit (ASIC).
- PLD's consists of configurable logic blocks and flip-flops linked together with programmable interconnect.
- Hierarchy of programmable interconnect allows the blocks to be "wired together" — somewhat like a one-chip programmable breadboard.
- PLDs can be used to implement any logical function that an ASIC could perform.

Semi-Custom: PLDs

□ Advantages:

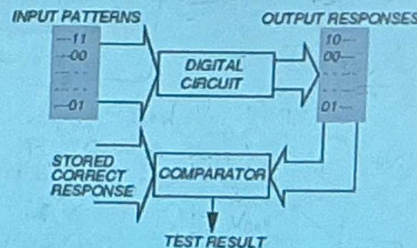
- ✓ Less time to market.
- ✓ The ability to update the functionality after shipping.
- ✓ Partial re-configuration of the portion of the design.
- ✓ The low non-recurring engineering costs relative to an ASIC design.

□ Disadvantages:

- ✓ Overall performance is satisfactory, but not excellent if compared to ASICs.
- ✓ Requires more area

Design Quality:

- Testability
- Manufacturability
- Reliability
- Technology updateability



CAD Technology

- **Synthesis tools**
 - ✓ VHDL
 - ✓ Verilog
- **Layout tools**
 - ✓ Microwind
 - ✓ Magic
 - ✓ Tanner
 - ✓ Synopsis
- **Simulation and Verification**
 - ✓ SPICE
 - ✓ HSPICE

DATE- 03/09/2024

Verilog HDL Overview

What is HDL?

- It is a specialized computer language
- Used to describe the structure and behavior of electronic circuits
- HDLs include the notion of time
- HDLs support Concurrency

Verilog

- Verilog is a Hardware Descriptive Language (HDL) standardized as IEEE 1364-2001
- Verilog is used to describe the low level hardware
- Syntax is similar to that of C-Language

Verilog versus C

Verilog	C
A Hardware Description Language(HDL) used to model electronic system	A general purpose programming language that allows structured programming
HDLs have a notion of time	Do not have notion of time
Helps to design and describe digital system	Helps to build operating systems, databases, compilers, interpreters etc.
Files have .v file extension	Files have .c file extension

VHDL

- VHDL stands for VHSIC (very high speed integrated circuit) Hardware Descriptive Language
- It can model the behavior and structure of digital system at multiple abstraction levels
- VHDL is strongly typed and very deterministic

Verilog versus VHDL

Verilog	VHDL
Used to model electronic system	Used in electronic design automation to describe digital and mixed signal system
Verilog is weakly typed	VHDL is strongly typed
Based on C language	Based on Pascal and Ada language
Case sensitive	Not Case sensitive
Simple data types	More complex data types
Verilog is newer than VHDL	Older than Verilog
Less complex	More complex

What is Verilog HDL ?

- **Hardware Description Language**
- **Mixed level modeling**
 - Behavioral
 - Algorithmic (like high level language)
 - Register transfer (Synthesizable)
 - Structural
 - Gate (AND, OR)
 - Switch (PMOS, NMOS, JFET)
- Single language for design and simulation
- Built-in primitives and logic functions
- User-defined primitives
- Built-in data types
- High-level programming constructs

Verilog Level of Abstraction

Objective

- **Switch Level**
- **Gate Level**
- **Data Flow Level**
- **Behavioral Level**

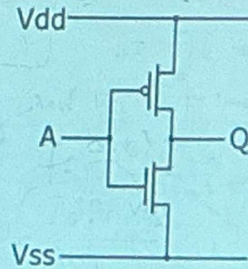
Switch Level

- At this level a module can be implemented in terms of switches
- Here nmos and pmos are used as switches for the design

Syntax: mos_name instance_name (output, data , control)

Example:

```
module inverter(Q,A);  
  input A;  
  output Q;  
  supply1 vdd;  
  supplyt0 vss;  
  
  pmos p (Q,vdd,A);  
  nmos n (Q,vss,A);  
  
endmodule
```



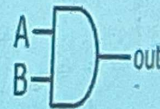
Gate Level

- At this level module is implemented in terms of logic gates
- Gate Level is the lowest level of abstraction
- Basic logic gates are available as predefined primitives

Syntax: Primitive_name instance_name(output , inputs)

Example:

1. and G1(out,A,B);



2. nand G2(Y,A,B);



3. or G3(out,A,B);



Data Flow Level

- Register Transfer Level
- At this level, module is designed by specifying the data flow
- signals are assigned by the data manipulating equations
- Design is implemented using continuous assignments
- All such assignments are concurrent in nature

- “assign” keyword is used

Example:

```
assign z = x & y ;  
assign p = q | r ;  
assign z = ~y ;
```

Behavioral Level

- This is the highest level of abstraction provided by HDL
- Behavioral level describes the system by its behavior
- Different elements like function, task, and blocks can be used
- Two important constructs under this level are initial and always

Modules and Instantiation

Modules

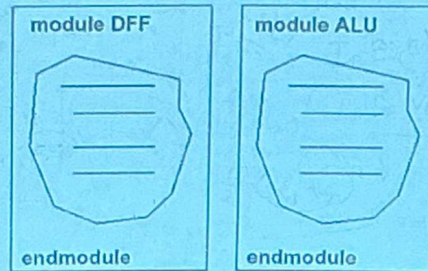
- A module is the basic building block in Verilog.
- A module can be an element or a collection of lower-level design blocks.
- It provides the required functionality to the higher-level block through its port interface but hides the internal implementation.
- It provides the flexibility to the designer to modify the module internals without affecting the rest of the design.

Verilog Framework:

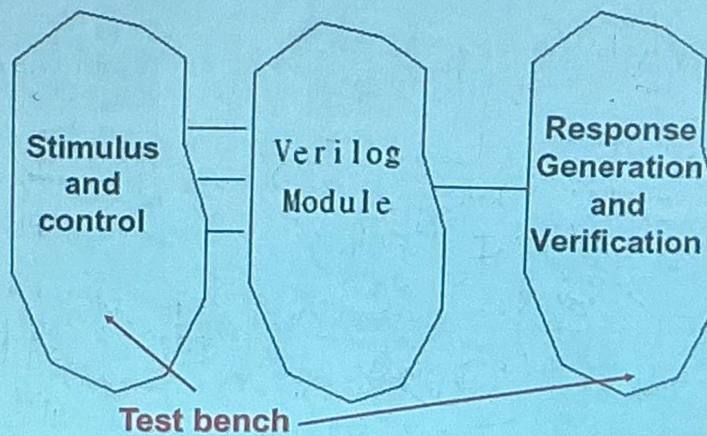
```
module module_name (x,y,z);  
    input x,y;  
    output z;  
    statements ;  
    ..... ;  
    ..... ;  
endmodule
```

Basic unit --Module

```
module module_name (port_name);  
    port declaration  
    data type declaration  
    module functionality or structure  
endmodule
```



Overview of Verilog Module



Module Instantiation

- Instantiation allows the creation of hierarchy in Verilog description.
- It is a process of creating object from a module template and the objects are called instances.
- In Verilog, nesting of module is illegal.
- One module definition cannot contain another module definition.
- we can incorporate the copies of other module by instantiating them.

```

module mux_4to1 (i0,i1,i2,i3,s1,s0,out);
input i0,i1,i2,i3,s1,s0;
output out;
wire x1,x2;

mux_2to1 m1 (i0,i1,s1,x1);
mux_2to1 m2 (i2,i3,s1,x2);
mux_2to1 m3 (x1,x2,s0,out);

endmodule

```

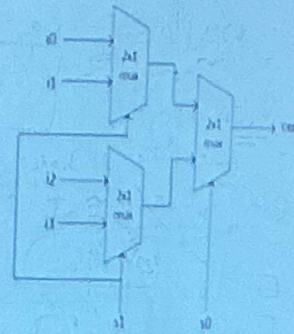
04/09/2024

Example: 4*1 mux using 2*1 mux

```

module mux_2to1 (i0,i1,sel,out);
input i0,i1,sel;
output out;
always@(i0,i1,sel)
begin
if(sel)
out = i1;
else
out = i0;
end
endmodule

```



$$HA = A \oplus B$$

$$Carry = AB$$

(Half Adder)

```

module HA_1 (Out, Cin, A, B);
input (A, B);
output (Out, Cin);
assign Out = A ^ B;
assign Cin = A * B;
end module.

```

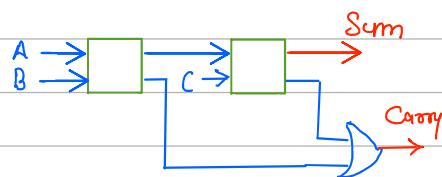
(Full adder)

```

module FA_1 (Sum, Carry, A, B, C);
input (A, B, C);
output (Sum, Carry);
assign Sum = A ^ B ^ C;
assign Carry = (A & B) | (B & C) | (C & A);
end module.

```

HA	BVREDDY
1. Logical expression for Sum = $A \oplus B$ 2. Logical expression for Carry = AB 3. Minimum number of NAND Gates = 5 4. Minimum number of NOR Gates = 5	
FA	
1. Logical expression for Sum = $A \oplus B \oplus C$ 2. Logical expression for Carry = $AB + (A \oplus B)C$ 3. Minimum number of NAND Gates = 9 4. Minimum number of NOR Gates = 9	



Comments

Comments are basically used for documentation. It also improves the readability of the code.

There are two ways to write the comment:

- Single-line comment- It starts with "//". Verilog skips from that point to the end of the line.
- Multiple-line comment- It starts with "/*" and ends with "*/". Multiple-line comment cannot be nested.

Example:

```
assign z = x ^ y; // z is equal to x^y
assign r = p & q; /* This is multiline
comment */
```

Latch

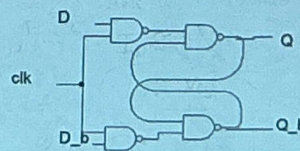
```
module Latch ( Q,Q_b,D,D_b,clk );
//port declaration
output Q,Q_b;
input D,D_b,clk;
reg Q,Q_b;

always@(D or D_b or clk or
        Q_b or Q)

begin
Q=(D&clk)|(~clk&Q);
Q_b=D_b&clk|~clk&Q_b;
end

endmodule
```

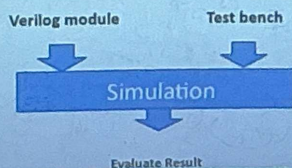
$$Q = D \cdot clk + \sim clk \cdot Q$$
$$Q_b = D_b \cdot clk + \sim clk \cdot Q_b$$



Simulation

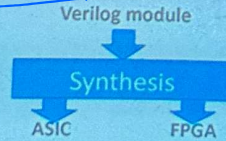
*Simulation is used to verify the functionality of the digital design that is modeled using HDL like Verilog.

*For simulation purpose we apply different input stimulus to the design at different time, to check whether the RTL code behaves in intended way or not.



Synthesis

- Synthesis is a process in which the digital design that is modeled using HDL is translated into an implementation consisting logic gates.
- It will just make an optimal design based on the working strategy which we are using and also give the consumption of resources as available.



Data Types Introduction

- Data type is a classification that specifies which type of value can be assigned to a variable.
- What type of mathematical operation can be applied to the data type

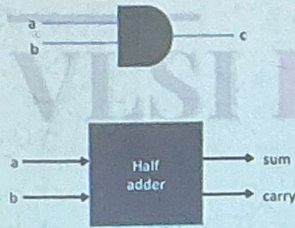
Register Data Type

- Register represents data storage elements
- It is a variable that can hold a value
- It is commonly declared by the keyword "reg"
- The default value of reg data type is X
- Register represents a class of data types such as reg, integer, real, time etc.

Net Data Type

- Nets represent connections between hardware elements.
- It must be continuously driven i.e. cannot be used to store the values
- Nets are declared primarily with the keyword "wire"
- The default value of net is Z
- Net represents a class of data types such as wire, wand, wor, tri, triand, trior, trireg etc.

- Consider the below figure, Net c is connected to the output of and gate.



```
module half_add (a,b,sum,carry);
  input a, b;
  output sum, carry;
  wire sum, carry;

  assign sum = a ^ b ;
  assign carry = a & b ;
endmodule
```



$$Y = (A \& B) \& (C \mid D)$$

```
module use_of_wire (y,A,B,C,D);
  input A,B,C,D;
  output y;
  wire y;

  assign y = A & B;
  assign y = C \mid D;
endmodule
```

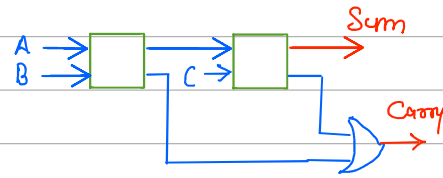
```
module use_of_wire (y,A,B,C,D);
  input A,B,C,D;
  output y;
  wand y;

  assign y = A & B;
  assign y = C \mid D;
endmodule
```

```

module FA_1 (Sum, Carry, A, B, C);
    input (A, B, C);
    output (Sum, Carry);
    assign Sum = A ^ B ^ C;
    assign Carry = (A & B) | (B & C) | (C & A);
end module.

```



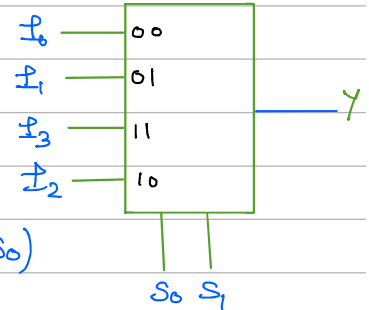
4x1 Mux —

Data-Flow —

```

module a4_1 MUX (Output = y, input = I0, I1, I2, I3, S1, S0)
    assign y = ~S0 ~S1 I0 + ~S0 S1 I1 + S0 S1 I2 + S0 ~S1 I3;
endmodule

```

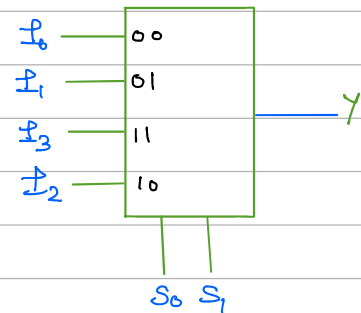


Behavioural —

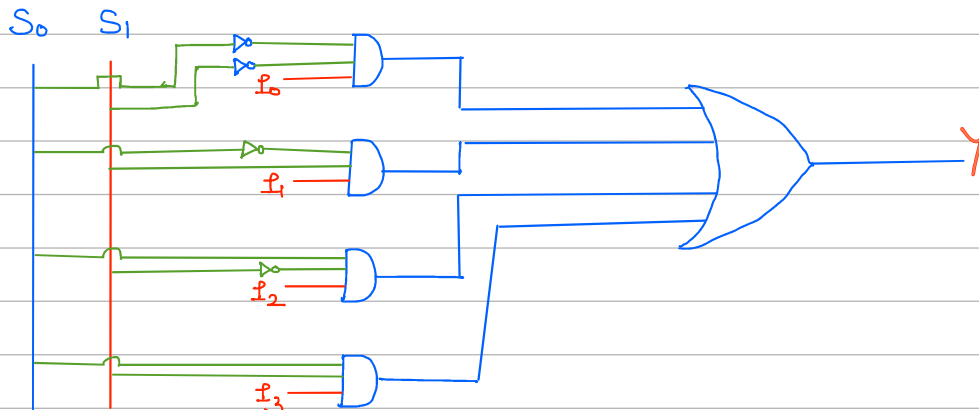
```

module a4_1 MUX (Output = reg y, input = S1, S0, I0, I1, I2, I3)
    always @ (*) begin
        if (S0 == 1'b0 & S1 == 1'b0)
            y = I0;
        else if (S0 == 1'b0 & S1 == 1'b1)
            y = I1;
        else if (S0 == 1'b1 & S1 == 1'b0)
            y = I2;
        else
            y = I3;
        #10;
    end
endmodule

```



$$y = \bar{S}_0 \bar{S}_1 I_0 + \bar{S}_0 S_1 I_1 + S_0 \bar{S}_1 I_2 + S_0 S_1 I_3$$



10/09/2024

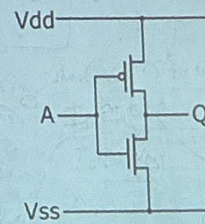
Example:

```
module using_wire_supply (Y,A,B,C,D);  
  input A,B,C,D;  
  output Y;  
  
  supply1 vdd;  
  supply0 gnd;  
  
  or G1 (x1, A, gnd);  
  and G2 (x2, B, vdd );  
  xor G3 ( Y,x1,x2);  
  
endmodule
```

Syntax: mos_name instance_name (output, data , control)

Example:

```
module inverter(Q,A);  
  input A;  
  output Q;  
  supply1 vdd;  
  supply0 vss;  
  
  pmos p (Q,vdd,A);  
  nmos n (Q,vss,A);  
  
endmodule
```



Various "Net" data types are supported in verilog for synthesis:

-wire, wor, wand, supply0, supply1 etc.

- "wire" and "tri" are equivalent
(when there are multiple drivers driving them, the driver outputs are shortened together)
- "wor" and "wand" inserts or and and gate respectively at the connection
- To model the power supply connection "supply0" and "supply1" are used

Values and Signal Strength

- Verilog supports 4 value levels and 8 strength levels to model the functionality of real hardware

Value Levels	Condition in Hardware Circuits
0	Logic zero, false condition
1	Logic one , true condition
X	Unknown Logic value
Z	High impedance, floating state

Signal Strength

Strength Level	Type	Degree
supply	Driving	Strong
strong	Driving	
pull	Driving	
large	Storage	
weak	Driving	
medium	Storage	
small	Storage	
highz	High Impedance	Weak

supply1 •
pull1 •

• supply1

strong1 •
strong0 •

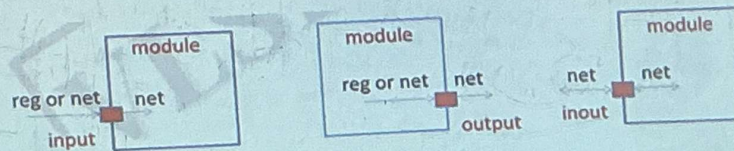
• X

- If two signals of unequal strength are driven on a wire, the stronger signal prevails.
- If two signals of equal strengths are driven on a wire, the result is unknown.

Port Assignments

Inputs, Outputs and Inouts

- Input- internally net, externally reg, or net
- Output- internally reg or net, externally net
- Inout- only wire data type



Numbers

Syntax: <size>'<base> <number>

```
1'b1      //logic-1 (1-bit)
8'h21     //8 bit number 0010 0001
12'ha35   //12 bit number 1010 0011 0101
12'o42xx  //12 bit number 100 010 xxx xxx
4'd1      //4 bit number 0001
35        //signed number , in 32 bits(size is not specified)
```

100011

Handwritten binary-to-decimal conversion table:

2	3	5
2	1	1
2	8	1
2	4	0
2	2	0
2	1	0

Numbers

Binary – b or B
Octal – o or O
Decimal – d or D
Hexadecimal – h or H

Use **underscores** to improve readability

32'h64_ab can also be written as

32'b 0110_0100_1010_1011

Zero, X and Z extension

4'bX0 // XXX_0

5'd1 // 0000_1

16'hz6 // ZZZZ_ZZZZ_ZZZZ_0110

Verilog supports following register data types-

- reg (most widely used)
- integer (used for loop counting)
- real (used to store floating point numbers)
- time (keeps track of simulation time)

EXAMPLE:

```
reg count;  
//single bit register variable  
  
reg [7:0] bus;  
//8 bit bus
```

```
reg reset;  
initial  
begin  
reset = 1'b0;  
#10 reset = 1'b1;  
#10 reset = 1'b0;  
end
```

24-Sept-2024

Verilog supports following register data types-

- **reg** (most widely used)
- **integer** (used for loop counting)
- **real** (used to store floating point numbers)
- **time** (keeps track of simulation time)

Register data type

- Register represent data storage elements
- It is a variable that can hold a value
- It is commonly declared by the keyword "**reg**"
- The default value of reg data type is X
- Register represents a class of data types such as reg, integer, real, time etc

EXAMPLE:

```
reg count;  
//single bit register variable  
  
reg [7:0] bus;  
//8 bit bus
```

```
reg reset;  
initial  
begin  
reset = 1'b0;  
#10 reset = 1'b1;  
#10 reset = 1'b0;  
end
```

Integer Data Type

- An integer is a general purpose register data type
- Declared by the keyword **"integer"**
- Store the values as signed quantities
eg. 0, 4, -17, -148, 57 etc
- Generally used for counting purpose
- Synthesizable in nature
- Default value is X
- Default width is 32 bits

EXAMPLE:

```
integer count, new_num;  
initial  
begin  
  count = 4;  
  new_num = -17;  
end
```

Real Data Type

- Declared by the keyword **"real"**
- Allows decimal and scientific notation
eg. 0.3, 4.63, -17.5, -1.485 etc
- Not synthesizable
- Default value is 0.0
- Rounded off to the nearest integer when it is assigned to an integer

EXAMPLE:

```
real count, new_num;  
initial  
begin  
count = 4.76;  
new_num = -1.3;  
end
```

```
integer a;  
real b;  
initial  
begin  
b = -6.5;  
a = b;  
end  
// a will be assigned the -7
```

Time Data Type

- A special time register data type is used to store the simulation time
- Declared by the keyword **"time"**
- Time variables are unsigned in nature
- Width is implementation-specific but is at least 64 bits
- Not synthesizable (useful for testbench)
- Default value is X

EXAMPLE:

```
time new_sim_time;  
initial  
begin  
new_sim_time = $time;  
//save the current simulation time  
end
```

Vectors

- nets or reg data types can be declared as vectors (multiple bit widths)
- If bit width is not specified, default size is scalar(1-bit)
- Vectors represent buses
- Vectors are declared by specifying the range [r1 : r2] where r1 is always the most significant bit (MSB) and r2 is the least significant bit (LSB).

EXAMPLE:

```
wire x,y,z;           //single bit variable
wire [15:0] data;     //16-bit data
wire [0:31] sum       //MSB is sum[0], LSB is sum[31]
reg clock;           //scalar register, default
reg [7:0] d1,d2,d3;   //3 buses of 8-bit width
reg [1:40] bus;       //vector register, 40-bit bus
```

Arrays

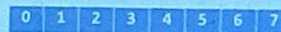
- In Verilog arrays are allowed for reg, integer, time, real, vector register data types
- Multi-dimensional arrays can be declared with any number of dimensions

```
reg [7:0] register_bank [15:0]; // 16 8-bit registers
reg num [31:0] ;                // array of 32 one-bit numbers
```

EXAMPLE:

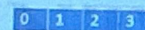
1. reg rega [0:7];

//an array of 8, 1-bit registers



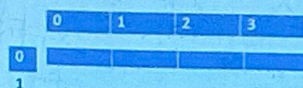
2. reg arraya [0:1] [0:3];

//Two dimensional array of 1-bit registers



3. wire arrayw [0:7] [0:3];

//Two dimensional array of wires



0
1
2
3
4
5
6
7

Memories

- Memories are modeled as a one-dimensional array of registers
- Used to model register files, ROMs and RAMs
- Each element of the array is known as an element or word
- These elements are addressed by a single array index
- Each element can be of one or more bits

EXAMPLE:

```
reg mem_1bit [0:1023];  
//Memory mem_1bit with 1K 1-bit words  
  
reg [7:0] membyte [0:1023];  
//Memory membyte with 1K 8-bit words
```

7 6 5 4 3 2 1 0

0

1

2

.

.

.

1023

Parameters

- Parameters cannot be used as variables, it is a constant value
- Parameter values for each module instance can be overridden individually at compile time
- This allows the module instances to be customized

```
parameter width_new = 8;  
//Defines width_new as a constant value 8  
  
parameter depth_new = 256;  
//Defines depth_new as a constant value 256
```

Strings

- Strings can be stored in reg
- The width of reg variables must be larger enough to hold the string
- Each string character takes up 8 bits(1 byte)
- A sequence of characters enclosed by double quotes(" ")

```
reg [8*10 : 1] string_value;    //Declaration  
string_value = "VLSI Point";    //assignment
```

Some special characters of string are newline, tabs and other displaying argument values.

Escaped characters	Characters displayed
<code>\n</code>	newline
<code>\t</code>	tab
<code>\%</code>	%
<code>\\</code>	\
<code>\"</code>	"

23-Sept-2024

Verilog provides various types of operators

- **Arithmetic Operators**
- **Logical Operators**
- **Bitwise Operators**
- **Equality Operators**
- **Relational Operator**
- **Reduction Operator**
- **Shift Operator**
- **Concatenation**
- **Conditional**

Arithmetic Operators

There are two types of arithmetic operators:

- **Unary-** The operators + and – can work as unary operators. They are used to specify the positive or negative sign of the operand.

e.g. `-4` // Negative 4

`+5` // Positive 5

- **Binary Operators-** Binary arithmetic operators are multiply (*), add (+), subtract (-), power (**) and modulus (%). Binary operators take two operands. If any operand bit has a value x, then the result of the entire expression is x.

Arithmetic Operators

Operator Symbol	Operation Performed	Number of Operands
+	Add	two
-	Subtract	two
*	Multiply	two
/	Divide	two
%	Modulus	two
**	Power(exponential)	two

Example: `A = 4'b0010; B = 4'b0011;`

A and B are register vectors.

`(D = 7; E = 4; F = 2)`

D, E and F are integers.

```
A * B // Multiply A and B. Evaluates to 4'b0110
D / E // Divide D by E. Evaluates to 1. Truncates any
fractional part.
A + B // Add A and B. Evaluates to 4'b0101
B - A // Subtract A from B. Evaluates to 4'b0001
F = E ** F // E to the power F, yields 16
```

Example: If any operand bit has a value x, then the result of the entire expression is x.

```
a = 4'b0001x ; b = 4'b1011 ;
sum = a + b ; // sum will be evaluated to the
value 4'bx
```

Modulus operators produce the remainder from the division of two numbers.

```
17 % 4 // Evaluates to 1
16 % 4 // Evaluates to 0
-7 % 2 // Evaluates to -1, takes sign of the first operand
7 % -2 // Evaluates to +1, takes sign of the first operand
```

Logical Operators

Logical operators are logical-and (&&), logical-or (||), logical-not (!). Operator ! is a unary operator. Operator && and || are binary operator. Logical operators always evaluate to a 1-bit value, 1(true), 0(false) or x (ambiguous).

Operator Symbol	Operation Performed	Number of Operands
!	logical negation	One
&&	logical and	two
	logical or	two

Example: A = 5; B = 0;

```
A && B // Evaluates to 0. Equivalent to (logical-1 && logical-0)
A || B // Evaluates to 1. Equivalent to (logical-1 || logical-0)
!A     // Evaluates to 0. Equivalent to not(logical-1)
!B     // Evaluates to 1. Equivalent to not(logical-0)
```

// Unknowns

```
A = 2'b0x; B = 2'b10;
A && B // Evaluates to x. Equivalent to (x && logical 1)
```

Bitwise Operators

It performs a bit by bit operation on two operands. It takes each bit in one operand and perform the operation with the corresponding bit in the other operand.

Operator Symbol	Operation Performed	Number of Operands
&	bitwise and	two
	bitwise or	two
~	bitwise negation	one
^	bitwise xor	two
^^ or ^^	bitwise xnor	two

Example:

X = 4'b1010, Y = 4'b1101, Z = 4'b10x1

```
~X // Negation. Result is 4'b0101
X & Y // Bitwise and. Result is 4'b1000
X | Y // Bitwise or. Result is 4'b1111
X ^ Y // Bitwise xor. Result is 4'b0111
X ^^ Y // Bitwise xnor. Result is 4'b1000
X & Z // Result is 4'b10x0
```

Equality Operators

Equality operators are:

- logical equality (==)
- logical inequality (!=)
- case equality (===)
- case inequality (!==)

It returns logical value 1 if true and 0 if false.

The important difference between the logical equality operators and case equality operator is that logical equality do not include x and z however case equality includes x and z.

Interview
que
form
operators

Example: A = 4, B = 3

X = 4'b1010, Y = 4'b1101, Z = 4'b1xxz

M = 4'b1xxz, N = 4'b1xxx

A == B // Results in logical 0

X != Y // Results in logical 1

X == Z // Results in x

Z === M // Results in logical 1 (all bits match, including x and z)

Z === N // Results in logical 0 (least significant bit does not match)

M !== N // Results in logical 1

Relational Operator

Relational operators are :

- greater than (>)
- less than (<)
- greater than or equal (>=)
- less than or equal (<=)

It returns a logical value of 1 if the expression is true and 0 if the expression is false.

Example: A = 4, B = 3

X = 4'b1010, Y = 4'b1101, Z = 4'b1xxx

A <= B // Evaluates to a logical 0

A > B // Evaluates to a logical 1

Y >= X // Evaluates to a logical 1

Y < Z // Evaluates to an x

Reduction Operator

Reduction operators are:

- and (&)
- or (|)
- nand (~&)
- nor (~|)
- xor (^)
- xnor (~^, ^~)

It takes only one operand. Reduction operator performs a bitwise operation on a single vector operand and yield a 1-bit result.

Example: $X = 4'b1010$

&X //Equivalent to $1 \& 0 \& 1 \& 0$. Results in $1'b0$

|X //Equivalent to $1 | 0 | 1 | 0$. Results in $1'b1$

^X //Equivalent to $1 \wedge 0 \wedge 1 \wedge 0$. Results in $1'b0$

A reduction xor or xnor can be used for even or odd parity generation of a vector.

Shift Operator

Shift operators are:

- right shift ($\gg$)
- left shift ($\ll$)
- arithmetic left shift ($\lll$)
- arithmetic right shift ($\ggg$)

Regular shift operators shift a vector operand to the right or the left by a specified number of bits. When the bits are shifted, the vacant bit positions are filled with zeros.

Arithmetic shift operators use the context of the expression to determine the value with which to fill the vacated bits.

Example: $X = 4'b1100$

$Y = X \gg 1$ //Y is $4'b0110$. Shift right 1 bit. 0 filled in MSB position.

$Y = X \ll 1$ //Y is $4'b1000$. Shift left 1 bit. 0 filled in LSB position.

$Y = X \lll 2$ //Y is $4'b0000$. Shift left 2 bits.

$Y = X \ggg 1$ //Y is $4'b1110$. Shift left 1 bits.

integer a, b, c (Signed data types)

a = 0, b = -10 (00111...10110 binary)

c = a + (b >>> 3) //Results in -2 decimal, due to arithmetic shift

Concatenation Operator

The concatenation operator (`{ , }`)

- Provides a mechanism to append multiple operands
- Operands must be sized
- Unsized operands are not allowed because the size of each operand must be known for computation of the size of the result
- Operands can be scalar nets or registers, vector nets or registers, bit-select, part-select, or sized constants

Concatenations are expressed as operands within braces, with commas separating the operands.

Example:

`A = 1'b1, B = 2'b00, C = 2'b10, D = 3'b110`

```
Y = {B, C} // Result Y is 4'b0010
Y = {A, B, C, D, 3'b001} // Result Y is 11'b10010110001
Y = {A, B[0], C[1]} // Result Y is 3'b101
Y = {A, 2{B}, 3{C}} // Result Y is 11'b10000101010
```

Conditional Operator

The conditional operator(`?:`) takes three operands.

Usage: `condition_expr ? true_expr : false_expr ;`

- The condition expression (`condition_expr`) is first evaluated.
- If the result is true (logical 1), then the `true_expr` is evaluated.
- If the result is false (logical 0), then the `false_expr` is evaluated.

Conditional operations can be nested.

```
assign out = (A == 3) ? ( control ? x : y ) : ( control ? m : n ) ;
```

Conditional Operator

The action of a conditional operator is similar to a multiplexer. Alternately, it can be compared to the if-else expression.

